HACKTRONIX 2.0

FINAL ROUND — PROBLEM STATEMENT COMPENDIUM

For Internal Participants Only

TRACK A
ROS2 & Robotics Simulation

TRACK B
Artificial Intelligence

TRACK A — ROS2 & ROBOTICS SIMULATION

Theme1:Robotics · FinalRoundProblemStatements

FINAL ROUND: 3D LiDAR-Based Shelter Damage Inspection System

Participants are tasked with designing and implementing a shelter inspection system capable of scanning rectangular shelters using a 3D LiDAR sensor, precisely localising structural damage, and classifying that damage by structural zone — **all without human intervention**.

1.1 Simulated Environment

Construct a Gazebo simulation environment containing two or more shelters (rectangular cuboids) with randomly generated structural damage — surface breaks, dents, holes, or deformations — varying in location, severity, and extent. At least one shelter model should be completely undamaged.

1.2 Linear Scanning Mechanism

The 3D LiDAR must be mounted on a fixed overhead rail platform spanning the open face of the shelter. The platform moves the sensor in a single straight line — entering from one end and exiting from the other — implemented as a rigid frame model with one **prismatic (sliding) joint** in Gazebo.

- Execute a scan achieving adequate coverage of the full shelter interior in a single linear pass.
- Accumulate LiDAR returns across the pass into a single coherent 3D point cloud.
- Operate without any human guidance after the scan command is issued.

Implementation hint: Define a static frame in SDF/URDF straddling the shelter entrance. Attach the LiDAR to a child link via a prismatic joint with position limits set to shelter depth. Use a ros2_control position interface or simple joint trajectory to command carriage position.

1.3 Damage Localisation

From the acquired point cloud, automatically detect and localise regions of structural damage. For each identified damage instance, produce:

- A precise 3D spatial location (centroid coordinates) within the shelter's reference frame.
- A confidence measure indicating certainty of the detection.
- A spatial extent descriptor (bounding volume or surface area of the affected region).

1.4 Damage Classification

Each detected damage region must be automatically classified into one of the following structural zones:

Zone Label	Definition
Face	Damage on a flat panel surface — a primary wall, roof, or floor — away from any edge or corner junction.

Zone Label	Definition
Edge	Damage along a structural edge where two faces meet — includes top rails, bottom rails, and vertical corner posts at their mid-spans.
Corner	Damage at or near a corner casting or a three-way junction of faces — the highest structural complexity zone.

3. Objectives

#	Deliverable	Description	Evaluation Criteria
O1	Damage Localisation	Detect and 3D-localise all damage instances, reporting centroid position and extent.	Spatial accuracy (distance error vs. ground truth), recall across damage instances
O2	Zone Classification	Correctly classify each detected damage region without human input.	Classification accuracy across zone types; handling of ambiguous or boundary cases

4. Technical Requirements & Constraints

4.1 Mandatory Constraints:

- The system must operate fully autonomously — no human interaction permitted after initialisation.
- The simulation environment must be built in Gazebo with ROS 2 as the middleware layer.
- All code, models, and configuration files must be version-controlled and reproducible.

4.2 Damage Generation:

Damage must be generated procedurally or randomly, not hand-placed. The damage generator must be capable of:

- Placing damage at any location on any surface (face, edge, corner).
- Varying damage severity independently of location.
- Generating multiple damage instances per shelter.
- Producing ground truth labels (location, zone, severity) for evaluation.

4.3 Output Format:

The system must produce a structured output report for each inspected shelter including: 3D centroid coordinates, structural zone classification label, severity rating, estimated repair cost, and shelter-level summary totals.

5. Deliverables

#	Deliverable	Description	Evaluation Criteria
D1	Gazebo World File	A runnable .world or .sdf containing ≥2 shelter models with procedurally generated damage and LiDAR rail platform.	Launches without errors; damage visually and geometrically present
D2	Scanning & Perception Pipeline	ROS 2 nodes implementing linear scan execution, point cloud accumulation, and damage detection.	Runs without human input after ros2 launch; produces labelled damage candidates

#	Deliverable	Description	Evaluation Criteria
D3	Classification Module	Software component that receives damage candidate point clouds and outputs zone classification labels.	Correct classification on ≥2 of the 3 zone types
D4	Source Repository	Version-controlled codebase with README sufficient to reproduce the full pipeline from a clean ROS 2 installation.	Judges can follow README and reproduce a run

6. Rules & Academic Integrity

- All code must be original or built on properly attributed open-source libraries.
- Pre-trained model weights are permitted provided the model architecture and training methodology are disclosed.
- Use of LLMs for code generation is permitted; teams must be able to explain every component.
- Sharing of solution approaches between teams during the event is not permitted.
- Judges' decisions on scoring are final.

7. Scope Guidance & Hints

- **The scanning mechanism is the easiest part.** A single prismatic joint in SDF commanded by one ROS 2 topic is all you need. Get it working in the first hour and move on.
- **Damage localisation is fundamentally a point cloud anomaly detection problem.** Think carefully about what a healthy shelter surface looks like geometrically.
- **Classification has both geometric and morphological components.** A hybrid approach combining both islikely more robust than either alone.
- **The simulation is your training data source.** The more varied and realistic your damage generator, the stronger a learned classifier you can train.

Track A — ROBOTICS QUALIFIER TASKS

Task1:TurtleBotNavigation · Task2:WaypointNavigation

Task 1: ROS2 Setup & TurtleBot Navigation

Install and configure ROS2 on the system, launch a TurtleBot simulation in Gazebo, and enable point-to-point navigation using the Nav2 stack.

Requirements:

- Proper ROS2 installation (Humble preferred).
- TurtleBot simulation running in Gazebo.
- Ability to send goal points and autonomously navigate to them.

Expected Demonstration:

- Successful launch of simulation.
- Robot navigating to at least two user-defined goal points.

Task 2: Repeated Waypoint Navigation using TurtleBot

Develop a looped waypoint navigation system where the TurtleBot navigates through a predefined set of waypoints and repeats the path continuously or for N cycles.

Requirements:

- Use Nav2 or custom ROS2 nodes.
- Robust navigation without manual intervention.

Expected Demonstration:

- Robot continuously following the waypoint loop.
- Stable localization and path planning.

TRACK B — ARTIFICIAL INTELLIGENCE

Theme2:AI · FinalRoundProblemStatements

FINAL ROUND: World Modeling for Autonomous Agents

Most AI agents today work by keeping every observation stuffed into an ever-growing context window. The longer they run, the slower, costlier, and more confused they get. The real problem is the **representation**. A capable agent does not need to remember every conversation — it needs a structured model of the world, updated continuously, queried efficiently, and compact enough to fit inside a small model's context.

1. Core Architecture

```
[WORLD INPUT]
↓
[EXTRACTOR] → structured description of what the agent currently sees
↓
[WORLD MODEL] ← you decide how to store and structure this
↓
[QUERY] → give the agent only what it needs right now
↓
[SLM AGENT] receives: { objective + current world slice } → outputs: action
↓
[UPDATER] → update the world model based on what changed
```

The agent is a **pure function**: next_action = f(objective, current_world_slice). It does not need to remember the past — the world model does that for it.

2. Getting Started

GPU and cloud API are **not permitted**. Run any small language model locally. Recommended starter models (all run on a standard laptop without GPU):

- qwen3.5:4b or qwen3.5:9b via LLama.cpp
- gemma2:2b via Ollama

Tool	Description	Link
Ollama	Easiest setup — one command to pull and run	ollama.com
llama.cpp	More control, quantized models on CPU	github.com/ggerganov/llama.cpp
LM Studio	GUI-based, good for quick experimentation	lmstudio.ai

3. Track 1 — Text World Agent

Using a text adventure game as your environment, build an agent that reads text descriptions of rooms, decides actions, and the game responds. Ensure the agent builds and maintains a structured model of the world as it explores.

What to Build:

- World Model — a structured, persistent store of rooms, objects, connections, and states.
- Updater — after every action, parse the game's response and update the world model.
- Query Layer — before every action, extract the relevant slice and hand it to the agent.
- Agent Loop — (objective + world slice) → action → game responds → update → repeat.

Environment	Description	Link
TextWorld	Microsoft Research — generate custom text game environments procedurally	github.com/microsoft/TextWorld
Frotz / Zork	Classic fixed text adventure. Good for benchmarking.	github.com/DavidGriffith/frotz
Custom	Build your own minimal environment for full control over game state.	—

What Good Looks Like:

- The agent completes multi-step objectives in an environment it has not fully seen before.
- The world model is structured, human-readable, and internally consistent.
- If the agent holds a wrong belief and receives contradicting information, it corrects itself.
- Performance scales gracefully as the explored world grows larger.

4. Track 2 — Vision World Modeler

Track 2 is an independent problem. Given a 1FPS video stream of an environment, build a system that watches the video and produces a structured, persistent, and self-correcting model of the world it observes. The world model must **get better with more observation**.

What to Build:

- Extractor — takes a single video frame and outputs a structured scene description.
- World Model — a persistent structured store that accumulates observations across frames.
- State Reconciliation — when a new frame contradicts an existing belief, update the model.
- Query Interface — given a frame index or location, return the current structured world state.

Recommended Vision Models (CPU-only):

- moondream2 — very lightweight, runs on CPU → github.com/vikhyat/moondream
- llava:7b via Ollama

---

5. Objectives

#	Objective	Description	Evaluation Criteria
O1	World Model Design	Structured, persistent, consistent, queryable — does not grow unboundedly.	Consistency; human-readability; no contradictions

#	Objective	Description	Evaluation Criteria
O2	Agent Behaviour (Track 1)	Complete multi-step objectives using only current world slice + goal. No conversation history.	Task completion; correct action selection; recovery from wrong beliefs
O3	Vision World Model (Track 2)	Watch video and produce structured world model growing more accurate over time.	Accuracy vs ground truth; frame consistency; uncertainty marking

6. Deliverables

#	Deliverable	Description	Acceptance Criteria
D1	Agent Loop (Track 1)	Working agent navigating text adventure with structured world model. No conversation history in context.	Completes ≥1 multi-room objective without human input
D2	World Model & Updater	Structured world store and update logic that writes to it after each action.	Consistent state across 20+ steps; human-readable; no contradictions
D3	Vision Extractor (Track 2)	Takes a video frame, outputs structured scene description. Runs locally.	Produces structured output for ≥5 distinct scene types
D4	State Reconciliation (Track 2)	Merges new visual observations with existing world model, handles uncertainty.	Correctly updates world state when same room seen twice with a change
D5	Source Repository	Version-controlled codebase with README sufficient to reproduce the full pipeline.	Judges can follow README and reproduce a complete run

7. Rules & Academic Integrity

- All code must be original or built on properly attributed open-source libraries.
- Pre-trained model weights are permitted provided the model architecture is disclosed.
- Use of LLMs for code generation is permitted; however, teams must be able to explain every component.
- Sharing of solution approaches between teams during the event is not permitted.
- Judges' decisions on scoring are final.

TRACK B — AI TASKS QUALIFIER TASKS

Task 1: BallDetection · Task2:MonocularFaceDistanceEstimation

Task 1: Ball Detection with Maximum F1 Score and Maximum FPS

Using a 2D camera, design a real-time ball detection system capable of identifying and localizing a ball under varied conditions while maximizing both detection performance and runtime speed.

Constraints:

- The model must maximize F1 Score.
- The system must run at the highest achievable FPS on the target hardware.
- Detection must remain stable and consistent across varied test environments.

Evaluation Metrics:

Metric	Role	Goal
F1 Score	Primary metric	Maximize the percentage of balls successfully detected
FPS	Secondary metric	Maintain real-time performance at highest feasible throughput
Combined Score	Overall	Reward both high detection quality and high runtime throughput

End Goal: Develop a monocular 2D camera-based ball detection system that achieves the best possible balance between maximum F1 Score and maximum FPS in real-time operation.

Task 2: Monocular Face Distance Estimation

Build a system that uses a single 2D camera image to estimate the face depth and the horizontal deviation angle from the camera center. Error of ±50–150 cm accuracy is acceptable.

Assumptions:

- A single monocular camera is used.
- The pinhole camera model is assumed.
- The real face width is approximately known (~0.14–0.16 m).
- The face center and face width in pixels can be detected.

Parameter	Meaning	Units / Form
(x, y)	Face center pixel	pixels
w_px	Face width in image	pixels
f	Focal length	pixels

Parameter	Meaning	Units / Form
(c_x, c_y)	Image center	pixels
W	Average real face width	meters

Mathematical Model:

```
Depth: Z = (f × W) / w_px
Angle: θ = arctan((x − c_x) / f) [radians or degrees]
```

Expected Output: (depth, θ) = (Z, θ) — estimated face distance and deviation angle from a single 2D image with approximate real-world accuracy.